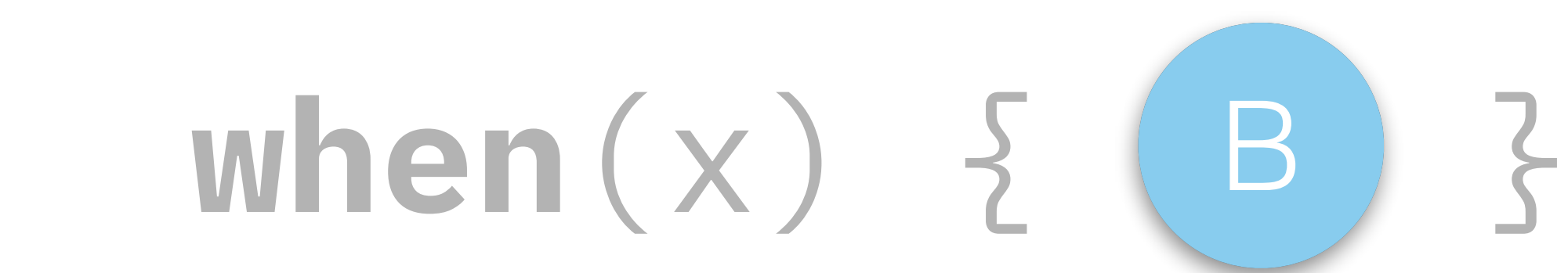
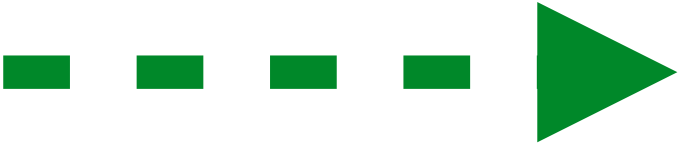

when(x) { B }

```
when(x) {
```

```
x.balance += 10;
```

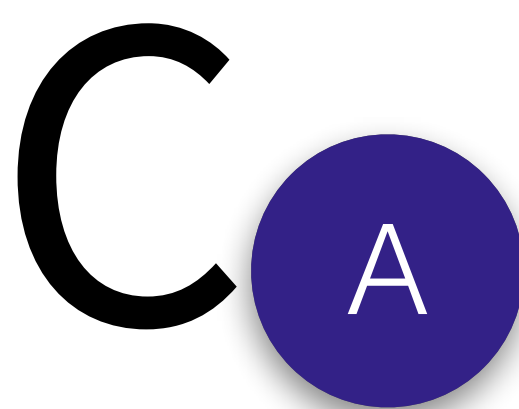


An Axiomatic Concurrency Model


42


S
A

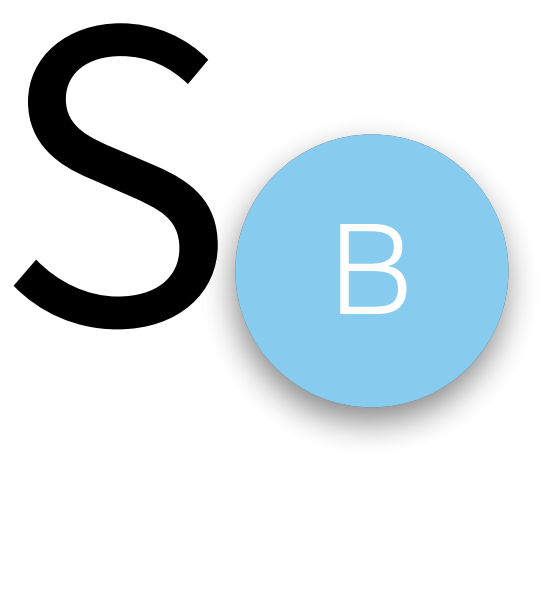

R
A


C
A

Each behaviour has three lifecycle events

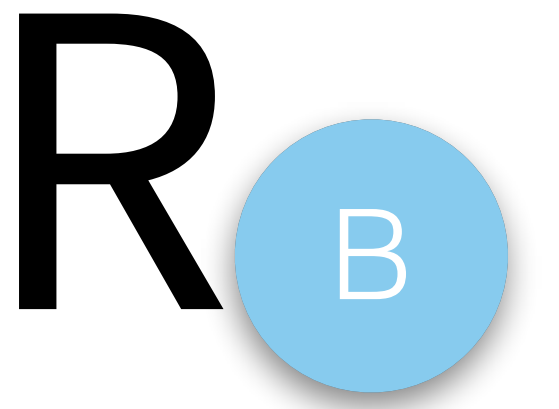
Executions are tuples of $(\mathbb{E}, po, co)$

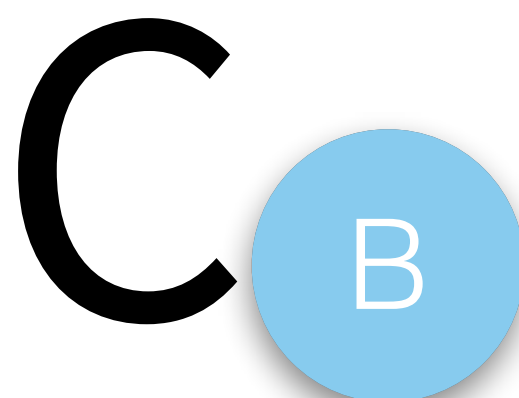
$\mathbb{E}$ are the events of the program


S
B

po

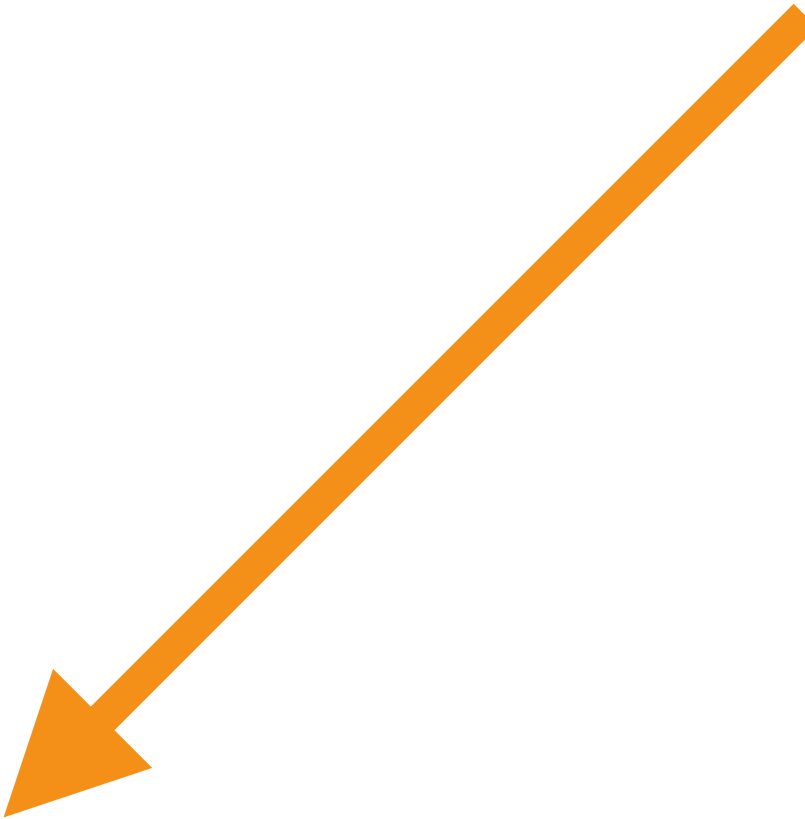

po


R
B


C
B

po

po is the program order

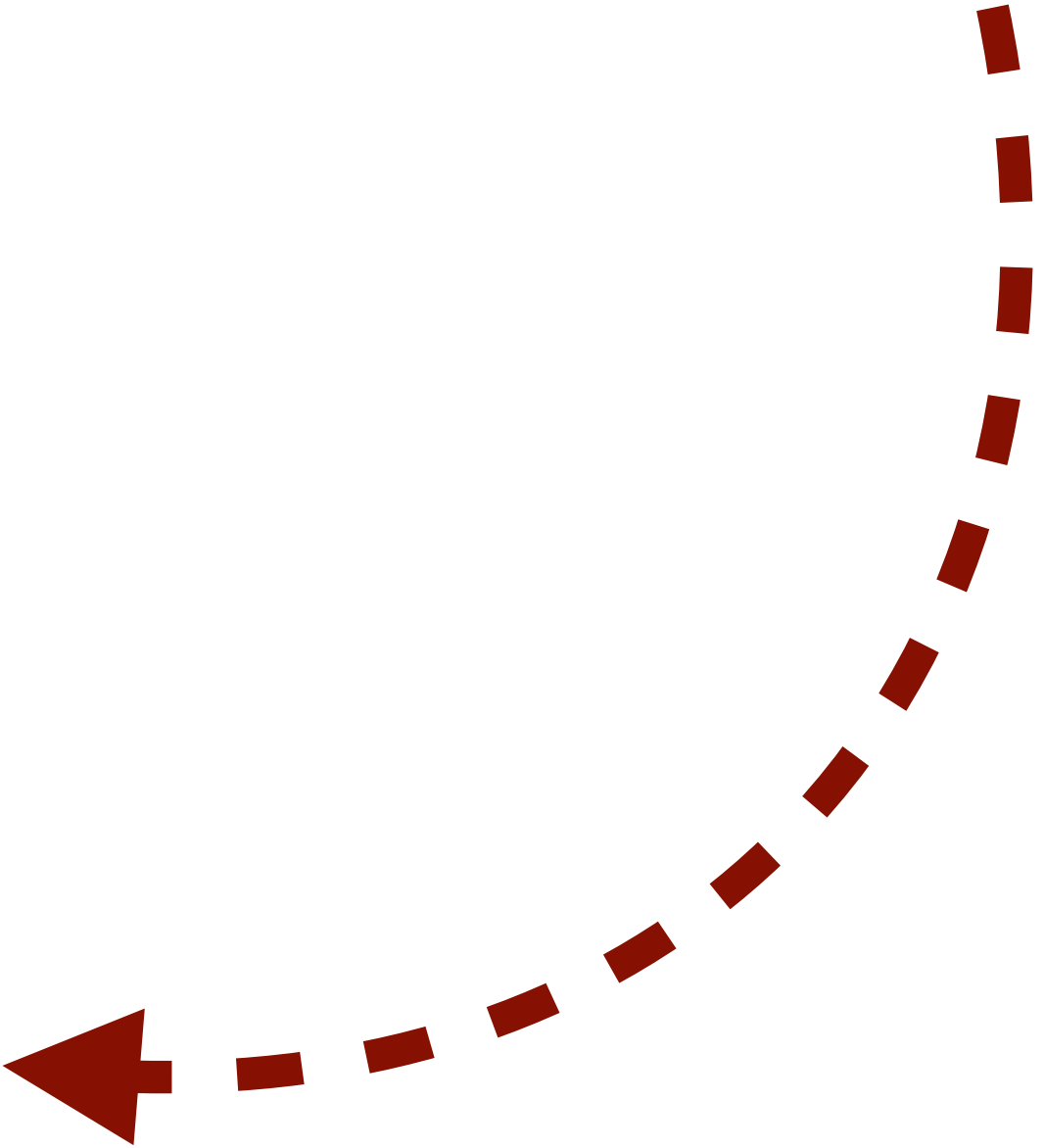


co is the cown order


CO
X

r

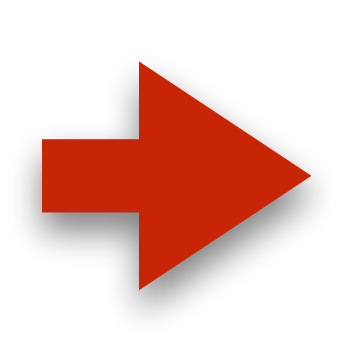
r is the derived run order

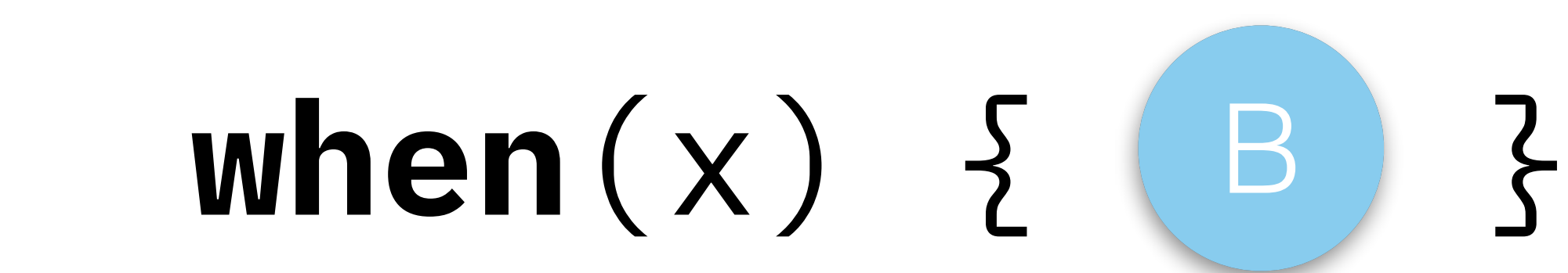



h b

hb is the derived happens before order

If behaviours share a cown and have a path between their spawns then complete happens-before run




when(x) { B }

```
when(x) {
```

```
x.balance += 10;
```

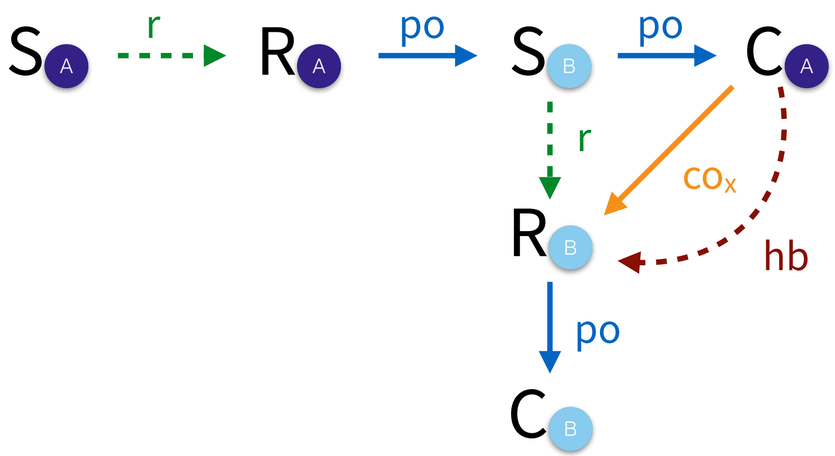
Valid if acyclic

An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Executions are tuples of $(\mathbb{E}, po, co)$

Each behaviour has three lifecycle events



$\mathbb{E}$ are the events of the program

po is the program order

co is the cown order

r is the derived run order

hb is the derived happens before order

Valid if acyclic

If behaviours share a cown and have a path between their spawns then complete happens-before run

Validating Causality

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

update(src, +100)

update(src, -100)